Deep Density Estimation for Audio Generation

Ankit Shrivastava
ashriva2@andrew.cmu.edu

Ryan Steed
rsteed@andrew.cmu.edu

Wen Wen
wenwen@andrew.cmu.edu

1 Introduction

At the I/O 2018, Google made headlines [Huffman, 2018] with its eerily realistic new voice assistant powered by a DeepMind advancement in text-to-speech (TTS) modeling, WaveNet [Oord et al., 2016]. Unlike previous TTS models, WaveNet can produce virtual voices with with different accents, intonations, and pitches. WaveNet powers Google's popular Google Assistant and provides audio for Google Translate. Combined with natural language processing, voice assistants using WaveNet voices can also call and interact realistically with humans. In a demo, Google's Duplex model, which combines WaveNet with other models for language comprehension, made autonomous phone calls to schedule an appointment at a hair salon, make a restaurant reservation, or ask for holiday hours [Leviathan and Matias, 2018]. In addition to personal assistant and customer service applications, TTS systems are useful for accessibility services, including text readers and electronic program guides.

WaveNet belongs to a popular class of generative models for text-to-speech applications. These generative models rely on *deep density estimation* to autoregressively predict audio samples, producing more natural-sounding speech and musical fragments. Deep density estimation seeks to model a distribution from a set of examples. For applications like audio generation, the goal is not only to model the distribution underlying audio samples, but also to *autoregressively* estimate new samples [Germain et al., 2015]. One common approach is to use an autoencoder model, which learns a hidden representation of the input that can then be transformed to reconstruct the input [Germain et al., 2015]. The reconstructed input models the original distribution.

We survey the use of similar deep density estimation techniques for audio generation and other time series tasks. Google's WaveNet follows the Masked Autoencoder for Distribution Estimation (MADE) [Germain et al., 2015], a masked version of the autoencoder approach that is computationally tractable in training. MADE has been applied to other domains as well, including image generation [Van Oord et al., 2016] and language modeling [Jozefowicz et al., 2016]. WaveNet can also be adapted to generate audio conditional on time-invariant covariates, such as speaker identity. But while training with WaveNet is computationally feasible, generation can be slow. Several theoretical improvements to WaveNet have attempted to mitigate this challenge; for example, Kingma et al. [2016]'s inverse autoregressive flow was implemented as Parallel WaveNet [Oord et al., 2018], which is slow to train but quick to generate new audio.

In contemporary audio generation techniques, there are clear trade-offs in both efficiency (training vs. generation) and architecture complexity. In this paper, we 1) summarize the theory behind masked autoencoders for distribution estimation (§3) and proposed improvements (§4), including variations on normalizing flows and new proofs; and 2) implement and compare two popular techniques for time series density estimation, autoregressive autoencoding and masked autoregressive flows. We also conduct a series of experiments in §5 to demonstrate the trade-offs of each model. §6 concludes.

2 Notation and Assumptions

Audio generation is typically accomplished through autoregressive density estimation. Given some training set of examples $\mathbb{X} = \{\boldsymbol{x}^{(i)}\}_{i=1}^{N}$ where each example $\boldsymbol{x}$ is a sequence $\boldsymbol{x} \in \mathcal{X}^d$, the goal of

Preprint. Under review.

density estimation is to estimate the underlying joint distribution $p(\boldsymbol{x})$ and output additional audio $\hat{x}_t$ based on the preceding sequence $x_{<t}$. By the probability product rule, the density can be expressed as a product of the marginal distributions conditional on $x_{<t}$

$$p(\boldsymbol{x}) = \prod_{t=1}^{d} p(x_t|x_{<t}) \tag{1}$$

Given an input sequence $\boldsymbol{x}$, the task of autoregressive density estimation is to estimate this marginal distribution $p(x_t|x_{<t})$, which can only be accomplished with a model that satisfies the autoregressive property.

Definition 1 (Autoregressive property)**.** The output $\hat{x}_t$ must be a function only of the preceding $x_{<t}$ and *not* the succeeding $x_{\geq t}$.

Audio generation techniques have improved drastically in recent years due to a series of improved deep density estimation techniques. In §3, we review the basic WaveNet model, a masked autoregressive autoencoder. More recently, another class of models have leveraged flow-based techniques to drastically improve training and generation eficiency. In §4 we review these flow-based improvements to the basic WaveNet model, including neural autoregressive flows and bipartite flows.

3 Autoregressive Autoencoders

The first set of highly successful models for audio generation are autoregressive autoencoders. WaveNet is one of these models. In this section, we describe how basic autoencoder models can be applied towards autoregressive audio generation. Theorems 1 and 2 show formally how this approaches satisfies the autoregressive property.

3.1 Autoencoders for Autoregression

Given data domain $\mathcal{X} = \{0, 1\}$, the goal is to learn a hidden representation of the input x using the following transformation:

$$h(\boldsymbol{x}) = g(b + W\boldsymbol{x})$$
$$\hat{\boldsymbol{x}} = \sigma(c + Vh(\boldsymbol{x}))$$

where g is some non-linear activation function and the sigmoid function $\sigma(a) = 1/(1 + \exp(-a))$. Assume some loss function (usually cross-entropy loss) $l(x)$ is used to train the model. How can we use an autoencoder for autoregressive distribution estimation, i.e. to get $p(x|x_{<t})$?

Theorem 1. *The cross entropy loss is a valid negative log likelihood for autoregression.*

Proof. Following Germain et al. [2015], recall the autoregressive formulation of the density in Equation 1. Then define $p(x_t = 1|x_{<t}) = \hat{x}_t$ such that $p(x_t = 0|x_{<t}) = 1 - \hat{x}_t$. Then we can turn the cross entropy loss into a valid negative log likelihood for autoregression

$$\begin{aligned}
\Rightarrow -\log p(\boldsymbol{x}) &= \sum_{t=1}^{d} -\log p(x_t|x_{<t}) \\
&= \sum_{t=1}^{d} -x_t \log p(x_t = 1|x_{<t}) - (1 - x_t) \log p(x_t = 0|x_{<t}) \\
&= \sum_{t=1}^{d} -x_d \log \hat{x}_t - (1 - x_t) \log(1 - \hat{x}_t) && \text{(by def p)} \\
&\equiv l(\boldsymbol{x}) && \text{(by def. cross entropy loss)}
\end{aligned}$$

□

3.2 Masked Autoencoders

A classical autoencoder is unsuitable for autoregression because it violates the autoregressive property. To modify the autoencoder to meet the autoregressive property, we need to zero out all the $x_{\geq t}$. This can be done simply with binary mask matrices M^W and M^V such that

$$h(\boldsymbol{x}) = g(b + (W \circ M^W)\boldsymbol{x})$$
$$\hat{\boldsymbol{x}} = \sigma(c + (V \circ M^V)h(\boldsymbol{x}))$$

where $M^W_{k,i} = 1_{m(k)\geq t} = \begin{cases} 1 & m(k) \geq t \\ 0 & \text{else} \end{cases}$ and $M^V_{i,k} = 1_{t>m(k)} = \begin{cases} 1 & t > m(k) \\ 0 & \text{else} \end{cases}$.

Theorem 2. *The masked autoencoder satisfies the autoregressive property.*

Proof. The autoregressive property holds if the connectivity matrix $M^{V,W}_{t',t}$ is strictly lower diagonal (then the matrix is 0 if $t' \leq t$ and all inputs $x_{>t}$ will be ignored).

$$M^{V,W}_{t',t} \equiv (M^V M^W)_{t',t} = \sum_{k=1}^{K} M^V_{t',k} M^W_{k,t} = \sum_{k=1}^{K} 1_{t'>m(k)} 1_{m(k)\geq t} \qquad \text{(by def.)}$$

Then we can see that the autoregressive property does hold - if $t' \leq t$ this expression is always 0. $m(k) \in [D-1]$ is the maximum number of inputs to which the kth unit can be connected. □

The mask is easily adapted to a neural network architecture: simply allow some connections and disallow others based on the masking matrix. The following section describes a few common implementations.

3.3 Architectures

PixelCNN [Van Oord et al., 2016]. In the image domain, consider the same problem formulation, but x is a sequence of pixels. Masking is accomplished with a masked convolution layer, and an LSTM is used for the autoencoder.

WaveNet [Oord et al., 2016] In the audio domain, consider the same problem formulation, but x is a waveform time-series. The WaveNet model [Oord et al., 2016] uses causal convolutions for masking (rather than masked convolutions) to ensure the model doesn't violate the data ordering and invalidate the autoregressive property. The activation function is $g \equiv z = \tanh(W_{f,k}\boldsymbol{x}) \circ \sigma(W_{g,k}\boldsymbol{x})$ where W is the convolution filter or mask. The choice of activation function is typically made experimentally. Typically, WaveNet is only used as a vocoder, for synthesizing speech. For end-to-end text-to-speech synthesis, WaveNet is combined with a larger model like Tacotron [Wang et al., 2017]. In §5, we demonstrate a practical implementation.

3.4 Conditional WaveNet

It may be desirable to condition WaveNet on other covariates, such as the speaker identity or language being spoken. With conditioning, WaveNet can produce audio to sound like a specific voice, a highly desirable feature. Consider the added problem where an additional input h is given, where h is the speaker identity or other covariate.

$$p(\boldsymbol{x}|h) = \prod_{t=1}^{D} p(x_t|x_{<t}, h)$$

Now the activation function is $g \equiv z = \tanh(W_{f,k}\boldsymbol{x} + V^T_{f,k}h) \circ \sigma(W_{g,k}\boldsymbol{x} + V^T_{g,k}h)$ where V is a linear projection broadcast over the time dimension. We can also do local conditioning where h_t is a time series. For this case, we upsample the time series h_t to the same resolution as the audio signal and then V is only a 1×1 convolution. For this case, the activation function is $g \equiv z = \tanh(W_{f,k}\boldsymbol{x} + V_{f,k}\tilde{h}_t) \circ \sigma(W_{g,k}\boldsymbol{x} + V_{g,k}\tilde{h}_t)$.

4 Flow-based Models

While autoregressive density estimators like WaveNet are very successful, they can be very slow for generation because they produce each new sample iteratively. Fortunately, autoregressive neural density estimators like MADE, PixelCNN, and WaveNet can often be expressed as invertible nonlinear transformations of the input with a simple Jacobian determinant. A set of flow-based density estimation models operate on this principle, transforming a simple initial density distribution into a more complex one by a series of transformations that can be computed *in parallel*. This approach accomplishes the same task as autoregressive autoencoding, but in a much more efficient manner.

This section describes the basic theory behind flow-based approaches and their trade-offs. Theorem 3 provides a formal guarantee for autoregressive density estimation with normalizing flows. The most successful example of flows in audio generation is Parallel WaveNet, an inverse autoregressive flow model. Theorem 4 proves that distillation can be used to train such a model through parallel sampling.

4.1 Autoregressive Flows

To understand autoregressive flow-based models, recall that we are trying to find the maximum marginal likelihood of Equation 1. Let $p(\boldsymbol{x}, \boldsymbol{z})$ be the parametric model of the joint distribution of the observables $\boldsymbol{x}$ and latent variables $\boldsymbol{z}$. Then we are trying to maximize (over the training set $\mathbb{X}$)

$$\log p(\mathbb{X}) = \sum_{i=1}^{N} \log p(\boldsymbol{x}^{(i)})$$

Of course, the marginal in general is hard to compute for flexible generative models like neural nets. One solution is to have a parametric inference model $q(\boldsymbol{z}|\boldsymbol{x})$, then optimize the variational lower bound, or ELBO

$$\begin{aligned} \log p(\boldsymbol{x}) &\geq \mathbb{E}_{q(\boldsymbol{z}|\boldsymbol{x})}[\log p(\boldsymbol{x}, \boldsymbol{z}) - \log q(\boldsymbol{z}|\boldsymbol{x})] = \mathcal{L}(\boldsymbol{x}; \theta) \\ &\Leftrightarrow \mathcal{L}(\boldsymbol{x}; \theta) = \log p(\boldsymbol{x}) - KL(q(\boldsymbol{z}|\boldsymbol{x}) \| p(\boldsymbol{z}|\boldsymbol{x})) \end{aligned}$$

We need the inference model to be computationally tractable, but also flexible enough to match the true posterior $p(z|x)$. Often, neural networks are used for this inference model, resulting in a class of models known as Variational Autoencoders (VAEs) [Kingma and Welling, 2014]. VAEs are analogous to autoregressive autoencoders like WaveNet. The general idea is to use a change of variables formula to calculate $p(\boldsymbol{x})$ based on the Jacobian of the transformation f:

$$p(\boldsymbol{x}) = p(\boldsymbol{z}) \left| \det \left(\frac{\partial f^{-1}(\boldsymbol{x})}{\partial \boldsymbol{x}} \right) \right|$$

Normalizing Flows. The simplest approach to variational inference with flows. Start with a simply distributed latent random variables z_t and apply invertible parameterized transforms f_t. As long as the Jacobian determinant of the transformations can be computed,

$$\begin{aligned} z_0 \sim q(z_0|\boldsymbol{x}), \quad z_t = f_t(z_{t-1}, \boldsymbol{x}) \quad \forall t \in [d] \\ \log q(z_d|\boldsymbol{x}) = \log q(z_0|\boldsymbol{x}) - \sum_{t=1}^{d} \log \det \left| \frac{dz_t}{dz_{t-1}} \right| \end{aligned}$$

Normalizing flows are a limited family of transformations $f_t(z_{t-1}) = z_{t-1} + uh(w^\intercal z_{t-1} + b)$, where the second addend can be easily modeled with a multi-layered perceptron.

Masked Autoregressive Flows (MAF). MAF [Papamakarios et al., 2018] is a computationally neater approach than the masked autoencoding model described in §3.2. Though the problem formulation is different, the MAF also uses autoregressive neural network to represent the transforming variable. The transformation of variable for MAF is $z_t = x_t \cdot \sigma_t(x_{<t}; \vartheta) + \mu_t(x_{<t}; \vartheta)$, where ϑ can be parameterized by a similar neural network architecture. Since the goal is to model sequential data, the conditional dependence on previous observed sequence (autoregressive property) needs to be upheld. To this end, MAF uses a masking approach analogous to masked autoencoding. The difference is that

masked autoregressive flows operate on a set of invertible nonlinear transformations. A MAF with one layer is a MADE with Gaussian conditionals. The determinant of the Jacobian is then

$$\det\left(\frac{\partial f^{-1}(\boldsymbol{x})}{\partial \boldsymbol{x}}\right) = \prod_t \sigma_t(x_{<t};\vartheta)$$

In §5, we tried to implement MAF [**Code**: Papamakarios [2019]] for the powerHebrail [2012] dataset. Like WaveNet, MAF is fast for density estimation and slow for generating sequential data. This is because audio and time series generation is a sequential procedure.

Inverse Autoregressive Flows (IAF). Inverse autoregressive flows (IAF) are a variant of MAF but transform Z to X all at once, and take p iterations to do so. In this way, MAF reverses the trade-off: IAF is fast for audio generation but slow for density estimation. MAFs model the data as $p(z_t|x_{<t})$, where IAFs model it as $p(x_t|z_{<t})$. The difference is practical: IAF can output x_t in parallel given z_t, where MAF outputs z_t in parallel given x_t. The latter is more useful for autoregressive density estimation; the former for generation.

Following Kingma and Welling [2014], consider a variable $y = \{y_i\}_{i=1}^d$ with lower triangular Jacobian (otherwise, the autoregressive property would be violated). $[\mu(y), \sigma(y)]$ are the mean and standard deviation and can be predicted. To sample, we want to transform a simple noise vector $\epsilon \sim N(0, I)$ to y.

$$\begin{aligned} y_0 &= \mu_0 + \sigma_0 \circ \epsilon_0 \\ y_i &= \mu_i(y_{1:i-1}) + \sigma_i(y_{1:i-1}) \cdot \epsilon_i \end{aligned}$$

For variational inference this is not very useful, but we can invert such that $\epsilon = (y - \mu(y))/\sigma(y)$ and compute the Jacobian $\log\det\left|\frac{d\epsilon}{dy}\right| = \sum_{i=1}^D -\log\sigma_i(y)$. To fit a neural network, we can chain multiple of these transformations:

$$\epsilon \sim N(0, I)$$
$$z_0 = \mu_0 + \sigma_0 \circ \epsilon, \quad z_t = \mu_t + \sigma_t \circ z_{t-1}$$

The neural network is constructed with masking so the Jacobians are triangular. Then the density can be modeled as $\log q(z_d|x) = -\sum_{i=1}^D \left(\frac{1}{2}\epsilon_i^2 + \frac{1}{2}\log(2\pi) + \sum_{t=0}^d \log\sigma_{t,i}\right)$. IAFs can be instantiated as an autoregressive neural network similar to WaveNet's.

Neural Autoregressive Flows (NAF). There is a more general way to express masked and inverse autoregressive flows which allows more formal analysis. Huang et al. [2018] express MAF and IAF as instances of neural autoregressive flows (NAF) of the form $z_t = \text{DNN}(x_t; \phi = c(x_{<t}))$, where c is some autoregressive conditioner (e.g. $[m_t, s_t$ from the NN in IAF) and DNN is a deep neural network with weights and biases ϕ. Huang et al. [2018] (Theorem 3) show that NAFs can approximate any probability distribution over real vectors arbitrarily well given that the DNN has sufficient hidden outputs. Consider the deep sigmoidal flow (DSF) architecture $z_t = \sigma^{-1}(w^\intercal\sigma(ax_t + b))$ which can be characterized by the pre-logit layer of a neural net

$$\mathcal{S}(x_t, \mathcal{C}(x_{<t})) = \sum_{j=1}^n w_j(x_{<t})\sigma\left(\frac{x_t - b_j(x_{<t})}{\tau_j(x_{<t})}\right)$$

where b_j and τ_j are bounded and positive functions.

Theorem 3. *Let X be a random vector in $\mathcal{X} \subset \mathbb{R}^m$ and Z be a random vector in $\mathbb{R}^m$ with positive, continuous density distributions. Then there exists a sequence of transformations (F_n) parameterized by autoregressive neural networks*

$$F(x)_t = \sigma^{-1}(\mathcal{S}(x_t; \mathcal{C}_t(x_{<t})))$$

where $\mathcal{C}_t$ are functions such that $Z_n = F_n(X)$ converges in distribution to Z.

Proof. It can be shown [Huang et al., 2018] that the pre-logit of DSF $(\mathcal{S}_n)$ converges uniformly to $\sigma \circ G \circ H$, where H are the CDFs of X and G are the inverse CDFs of Z. Uniform convergence implies point-wise convergence, so $F_n = \sigma^{-1} \circ \mathcal{S}_n$ converges pointwise to $G \circ H$. Since $G(H(X)) = Z$ by Lemma 1, $Z_n \xrightarrow{d} Z$. □

Lemma 1. *Given any function $F : \mathcal{X} \to \mathbb{R}^m$ and sequence of functions F_n that converges pointwise to F, $Z_n = F_n(X) \xrightarrow{d} Z = F(X)$.*

Proof. Let f be a bounded, continuous function such that $f \circ F_n \to f \circ F$ pointwise by continuity. f is bounded so by dominated convergence $f(Z_n) \to f(F_n(X))$ in expectation. By Portmanteau's lemma this result holds for any f. □

4.2 Parallel WaveNet

Given the result above, we can use inverse autoregressive flows to make classical WaveNet (from §3) much faster [Oord et al., 2018]. Now, let $x = f(z)$ with a simple distribution $p_Z(z)$ such that the Jacobian can be computed as

$$\log p_X(\boldsymbol{x}) = \log p_Z(\boldsymbol{x}) - \log \left| \frac{d\boldsymbol{x}}{d\boldsymbol{z}} \right|$$
$$\log \left| \frac{d\boldsymbol{x}}{d\boldsymbol{z}} \right| = \sum_t \log \frac{\partial f(\boldsymbol{z}_{\leq t})}{\partial z_t}$$

The full Parallel WaveNet [Oord et al., 2018] model is then

$$\begin{aligned} \boldsymbol{z} &\sim p_Z(z) \\ x_t &= z_t \cdot s(z_{<t}, \theta) + \mu(z_{<t}, \theta) \\ \Rightarrow p(x_t | z_{<t}, \theta) &= \mathbb{L}(x_t | \mu(z_{<t}, \theta), s(z_{<t}, \theta)) \end{aligned}$$

where μ and s are the network outputs, like μ and σ from IAF.

Theorem 4. *The probability distillation loss given in Equation 2 can be estimated by sampling in parallel from the student model $p_S(X)$ [Oord et al., 2018].*

Proof. First, note that entropy term can be written in terms of samples z drawn from the initial, simple distribution.

$$E(P_S) = \mathbb{E}_{z \sim p_Z(z)} \left[\sum_{t=1}^{T} - \ln p_S(x_t | z_{<t}) \right] = \mathbb{E}_{z \sim p_Z(z)} \left[\sum_{t=1}^{T} \ln s(z_{<t}, \theta) \right] + 2T$$

Then note that the cross-entropy term can be written in terms of the entropy of two autoregressive distributions, a teacher and a student:

$$\begin{aligned} CE(P_S, P_T) &= \int_x p_S(x) \ln p_T(x) \\ &= \sum_{t=1}^{T} \int_x p_S(x) \ln p_T(x_t | x_{<t}) \\ &= \sum_{t=1}^{T} \int_x p_S(x_{<t}) p_S(x_{\geq t} | x_{<t}) \ln p_T(x_t | x_{<t}) \qquad \text{(chain rule)} \\ &= \sum_{t=1}^{T} \mathbb{E}_{p_S(x_{<t})} \left[\int_{x_t} p_S(x_t | x_{<t}) \ln p_T(x_t | x_{<t}) \int_{x_{>t}} p_S(x_{>t} | x_{\leq t}) \right] \\ &= \sum_t \mathbb{E}_{z \sim p_Z(z), x = g(z)} E(p_S(x_t | z_{<t}), p_T(x_t | x_{<t})) \end{aligned}$$

We can evaluate this expression by drawing multiple samples each time step from the student p_S, then compute p_T in parallel with the teacher. Computing $E(P_S)$ is trivial. □

Compared to WaveNet. While WaveNet and Parallel WaveNet model the same distributions, their inductive biases are different [Oord et al., 2018]. For example, autoregressive models can use a receptive field of 2 to model a Fibonacci sequence $f(k) = f(k-1) + f(k-2)$, but IAF would need at least k receptive fields.

Table 1: Complexity of flow-based approaches to density estimation, where n is the length of the input audio sample x, adapted from [Ping et al., 2020]. h is the height of the architecture.

Model	Complexity of $X \to Z$	Complexity of $Z \to X$	h
MAF	$O(1)$	$O(n)$	small
IAF	$O(n)$	$O(1)$	small
Bipartite flows	$O(1)$	$O(1)$	large
WaveFlow	$O(1)$	$O(h)$	small to large

Probability density distillation. Training directly with MLE is impractical: the inference procedure for the log likelihoods is slow (the opposite of regular WaveNet and MAF, which trains quickly but generates slowly). Oord et al. [2018] try multiple different loss functions (power loss, contrastive loss, perceptual loss), but the most efficient training procedure is probability density estimation: a way for Parallel WaveNet $p_S(x)$ to learn from regular pre-trained WaveNet $p_T(x)$ with a loss based on cross-entropy and entropy:

$$KL(P_S \| P_T) = CE(P_S, P_T) - E(P_S) \tag{2}$$

The divergence of this loss is zero when the student has learned the teacher distribution. This is similar to GANs, but not adversarial. Additionally, choosing a logistic mixture for the teacher output distribution makes the log teacher loss differentiable (proof excluded for space) [Oord et al., 2018].

4.3 Other Flows

Both IAF and MAF can be generalized as neural autoregressive flows (NAF) [Huang et al., 2018]. These models require a pretrained WaveNet teacher and have complicated training procedures and loss functions. Another set of flow-based models based on bipartite transformations, like WaveGlow [Prenger et al., 2019], use only a single network trained on a single cost function. But these models require more complex architectures and must be highly parameterized. Table 3 summarizes the complexity and size trade-off for all the flow-based models.

The recent WaveFlow [Ping et al., 2020] model attempts to unify these two flows, under which bipartite flows and autoregressive flows are both special cases. Unlike MAF and IAF, NAF (and the bipartite flows) squeeze the input 1D waveform into an h-row 2D matrix X. Z is sample from an isotropic Guassian distribution. Then $Z_{ij} = \sigma_i j(X_{<i}; \Theta) X_{ij} + \mu_{ij}(X_{<i}; \Theta)$. The Jacobian is $\det\left(\frac{\partial f^{-1}(X)}{\partial X}\right) = \prod_{i=1}^{h} \prod_{j=1}^{w} \sigma_{ij}(X_{<i}; \Theta)$ and the log likelihood can then be calculated in parallel and used for simple MLE: $\log p(X) = \sum_{ij} \left(\log \sigma_{ij}(X_{<i}; \Theta) - \frac{Z_{ij}^2}{2} - \frac{1}{2}\log(2\pi)\right)$. To generate audio, we use Z to compute X: $X_{ij} = \frac{Z_{ij} - \mu_{ij}(X_{<i}; \Theta)}{\sigma_{ij}(X_{<i}; \Theta)}$. These flows can be stacked and parameterized by a neural network, just like with IAF and Parallel WaveNet.

5 Experiments

5.1 Masked Autoregressive Flow, code reference: Papamakarios [2019]

Like WaveNet, MAF relies on the MADE architecture. There are multiple modules in MAF implementation. Each module has three main components: 1) MADE-based neural nets with masking features; 2) a batch normalsation layer; 3) a reverse module which basically reverses the order for each successive layer. The architecture and parameters for MADE are given in Table 2. The training result is shown in Figure 1.

5.2 WaveNet (code)

A tensorflow implementation of wavenets was trained with 1655 audio samples from LJSpeech data set on GPU. The model is trained for 27248 steps with adam, after which the cross-entropy training loss is 2.652. The architecture we finalize is the following(Table 2) A generated audio sample from the trained model can be access here. Although at this point, this sample is mostly mumbling,we can still recognize the tone of the speaker from the training set. To generate more realistic audio sample,

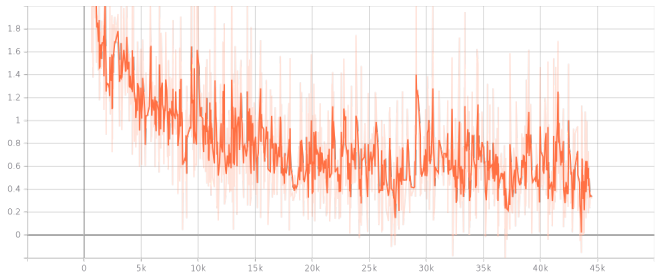

Figure 1: The training result for MAF. It is attached as proof that we tried to run MAF over power dataset. Later the plan was to use the model for audio generation and then compare the two models. However, training it for around 40K epochs took around 10 hr on psc cluster itself.

Table 2: **Architecture of MADE layer**

MaskedLinear1	in=6, out=100
MaskedLinear2	in=100, out=100
MaskedLinear2	in=100, out=12

Table 3: **Architecture of wavenets**

Parameters	Setting
Layer	10
Filter Width	2
Residual Channels	32
Dilation Channels	32
Quantization Channels	256
Skip Channels	512

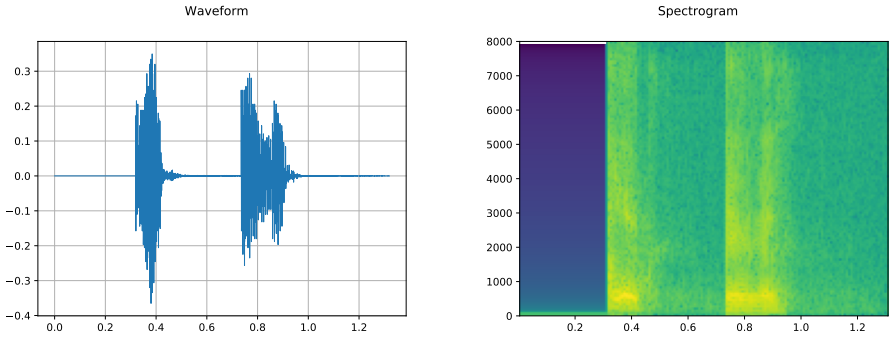

Figure 2: Waveform and specgram of generated 1-second audio sample

the model typically needs to train for 1.2M iterations. This observation agrees with our theory that though capable of generating raw audios, wavenets is very inefficient in terms of training. This gives us motivation to the development of parallel wavenets. Specifically, a flow-based network WaveGlow with invertible convolutions outperforms Wavenets in terms of mean opinion score(MOS) with only 580,000 training iterations.

6 Conclusion

In this paper, we survey two main approaches to audio generation: autoregressive autoencoding and flow-based models. We discuss the trade-offs between different flow-based approaches, including the two most common approaches, MAF and IAF. We survey several newer flow-based methods with provable guarantees and better computational efficiency. Finally, we demonstrate two practical implementations of time series density estimation using classical autoencoding and MAF, its flow-based analogue.

Division of Work

We roughly split the analysis and experimentation between the three versions of the WaveNet model. Each model solves roughly the same problem (neural density estimation), but with different theoretical techniques.

- Wen was responsible for training and experimenting Wavenets on 1655 audio data from LJSpeech dataset.[Ito and Johnson, 2017] She first try tunning the archiecture of Wavenets from a pytorch implementation but pure noise was generated after 10k iterations. After switching to a tensorflow implementation, decent results generated after 28k iterations.
- Shrivastava worked on masked autoregressive flow, he tries MAF on time series data set Hebrail [2012]. He was also responsible to compare experimentally wavenet with MAF, however due to large training and coding time the test of MAF on audio dataset remained incomplete.
- Steed was responsible for the conditional autoencoder model (Conditional WaveNet) and newer models (NAF, WaveFlow, WaveGlow). Because these models are just extensions of WaveNet and Parallel WaveNet, Steed was also responsible for writing the introduction and conclusion as well as formatting. The theoretical results (problem formulation and proofs) are adapted from a set of LaTeX notes written by Steed.

References

Scott Huffman. The future of the Google Assistant: Helping you get things done to give you time back, May 2018. URL https://blog.google/products/assistant/io18/.

Aaron van den Oord, Sander Dieleman, Heiga Zen, Karen Simonyan, Oriol Vinyals, Alex Graves, Nal Kalchbrenner, Andrew Senior, and Koray Kavukcuoglu. Wavenet: A generative model for raw audio. *arXiv preprint arXiv:1609.03499*, 2016.

Yaniv Leviathan and Yossi Matias. Google Duplex: An AI System for Accomplishing Real-World Tasks Over the Phone, May 2018. URL http://ai.googleblog.com/2018/05/duplex-ai-system-for-natural-conversation.html.

Mathieu Germain, Karol Gregor, Iain Murray, and Hugo Larochelle. Made: Masked autoencoder for distribution estimation. In Francis Bach and David Blei, editors, *Proceedings of the 32nd International Conference on Machine Learning*, volume 37 of *Proceedings of Machine Learning Research*, pages 881–889, Lille, France, 07–09 Jul 2015. PMLR. URL http://proceedings.mlr.press/v37/germain15.html.

Aaron Van Oord, Nal Kalchbrenner, and Koray Kavukcuoglu. Pixel recurrent neural networks. In *International Conference on Machine Learning*, pages 1747–1756. PMLR, 2016.

Rafal Jozefowicz, Oriol Vinyals, Mike Schuster, Noam Shazeer, and Yonghui Wu. Exploring the limits of language modeling. *arXiv preprint arXiv:1602.02410*, 2016.

Diederik P Kingma, Tim Salimans, Rafal Jozefowicz, Xi Chen, Ilya Sutskever, and Max Welling. Improving variational inference with inverse autoregressive flow. *arXiv preprint arXiv:1606.04934*, 2016.

Aaron Oord, Yazhe Li, Igor Babuschkin, Karen Simonyan, Oriol Vinyals, Koray Kavukcuoglu, George Driessche, Edward Lockhart, Luis Cobo, Florian Stimberg, et al. Parallel wavenet: Fast high-fidelity speech synthesis. In *International conference on machine learning*, pages 3918–3926. PMLR, 2018.

Yuxuan Wang, RJ Skerry-Ryan, Daisy Stanton, Yonghui Wu, Ron J Weiss, Navdeep Jaitly, Zongheng Yang, Ying Xiao, Zhifeng Chen, Samy Bengio, et al. Tacotron: Towards end-to-end speech synthesis. *arXiv preprint arXiv:1703.10135*, 2017.

Diederik P. Kingma and Max Welling. Auto-Encoding Variational Bayes. *arXiv:1312.6114 [cs, stat]*, May 2014. URL http://arxiv.org/abs/1312.6114. arXiv: 1312.6114.

George Papamakarios, Theo Pavlakou, and Iain Murray. Masked Autoregressive Flow for Density Estimation. *arXiv:1705.07057 [cs, stat]*, June 2018. URL http://arxiv.org/abs/1705.07057. arXiv: 1705.07057.

George Papamakarios. Masked autoregressive flow. https://github.com/gpapamak/maf, 2019.

Georges Hebrail. Individual household electric power consumption data set, 2012. https://archive.ics.uci.edu/ml/datasets/Individual+household+electric+power+consumption.

Chin-Wei Huang, David Krueger, Alexandre Lacoste, and Aaron Courville. Neural Autoregressive Flows. In *International Conference on Machine Learning*, pages 2078–2087. PMLR, July 2018. URL http://proceedings.mlr.press/v80/huang18d.html. ISSN: 2640-3498.

Ryan Prenger, Rafael Valle, and Bryan Catanzaro. Waveglow: A Flow-based Generative Network for Speech Synthesis. In *ICASSP 2019 - 2019 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*, pages 3617–3621, May 2019. doi: 10.1109/ICASSP.2019.8683143. ISSN: 2379-190X.

Wei Ping, Kainan Peng, Kexin Zhao, and Zhao Song. WaveFlow: A Compact Flow-based Model for Raw Audio. In *International Conference on Machine Learning*, pages 7706–7716. PMLR, November 2020. URL http://proceedings.mlr.press/v119/ping20a.html. ISSN: 2640-3498.

Keith Ito and Linda Johnson. The ljspeech dataset. https://keithito.com/LJ-Speech-Dataset/, 2017.